

Course Work Project Description and Rubric

Semester	202420		Division	CIS
Assessment title in Syllabus	Project		Program	IT and IS

Course Code		CIS 2423		
Course Title		Programming for Data Analytics		
CLOs	All CLOs		Accreditation Body	CAA & CIPS
Course Instructor	Asem Kasem		CRN	25871
Assessment Weight	40%		Submission Date	Week 14
For Group Work submissions an additional individual assessment will be conducted. Grades for the students in one group will vary based on the individual performance in the additional assessment.				

	Student Declaration: Academic Integrity Statement In accordance with the HCT Academic Integrity Policy - Students are required to refrain from all forms of academic integrity breaches as defined and explained by HCT. - A student found guilty of having committed acts of academic integrity breach(es) will be subject to the relevant sanctions as outlined by HCT. فأداة النزاهة الأكاديمية وفقاً لسياسة كليات التقنية العليا للنزاهة الأكاديمية - على الطلبة الالتزام بلوائح وقواعد النزاهة الأكاديمية، كما هو مبين وموضح في السياسات والإجراءات الخاصة بكليات التقنية العليا. - في حالة ارتكاب الطالب أي شكل من أشكال الإخلال بالنزاهة الأكاديمية، سيتعرض إلى العقوبات الموضحة في السياسات ذات الصلة. This assignment is entirely my own work except where I have duly acknowledged other sources in the text and listed those sources at the end of the assignment. I have not previously submitted this work to the HCT, or any other entity. I understand that I may be orally examined on my submission.
	Student (s) Signature: _____ _____

Student Name(s):	Hessa Khalfan	Maryam Ali	Meera Mohamed	Nourah Alghfeli
Student HCT ID(s):	H00535166	H005535836	H00535869	H00535834

For Examiner's Use Only

	Group (50%)					Individual (50%)		
CLO	1	2	3	4	Report Formatting	Oral Defense	Total	%

Marks Allocated	10	10	42	26	12	50	100	40
Marks Obtained								

Table of Contents

CLO 1 - Introduction to Data Analysis and Machine Learning	3
CLO 2 - Exploratory Data Analysis and Descriptive Statistics	4
Descriptive Statistics Report	10
1. Analysis by Category.....	10
A. Math Scores	10
B. Reading Scores.....	10
C. Writing Scores.....	10
2. Comparative Insights.....	10
3. Key Recommendations	11
4. Conclusion.....	11
CLO 3 - Machine Learning Model Development and Evaluation	16
CLO 3 - Version Control.....	32
Rubric.....	39
Group Project Rubric [Task-specific RUBRIC]	39
Individual Oral Presentation	43

CLO 1 - Introduction to Data Analysis and Machine Learning

1- Define the purpose of data analysis for the chosen dataset

The purpose of the data analysis for this dataset is to analyze, identify students' academic performance based on factors such as gender, parental education, lunch type, test preparation, and subject-wise scores. It aims to understand patterns and relationships between student demographics and external factors on their scores.

2- Identify and justify the type of programming used for data analysis

Python is utilized here as the preferred programming language due to its well-developed libraries such as **pandas/ numpy** (manipulation of data), **matplotlib** (visualization), and **scikit-learn** (machine learning). The ability of Python to efficiently handle large datasets and support machine learning and statistical methods makes it appropriate for the purpose of this work.

3- Identify the type and purpose of the machine learning algorithm to be implemented for the chosen dataset

The classification algorithm, for example, Logistic Regression can predate student scores based on independent variables. We can also classify students based on their performance level.

4- Identify and justify the independent and dependent variables for the chosen dataset.

Independent Variables (Predictors): These are factors that influence students' performance:

Gender (categorical)

Race/ethnicity (categorical)

Parental education level (categorical)

Lunch type (categorical)

Test preparation course completion (categorical)

Dependent Variables (Target): These are the outcomes we aim to predict:

Math score (numerical)

Reading score (numerical)

Writing score (numerical)

CLO 2 - Exploratory Data Analysis and Descriptive Statistics

5- Justify why you want to perform the descriptive analysis for the chosen dataset.

Descriptive analysis is important for understanding the dataset's characteristics before applying machine learning. It helps in:

- Identifying **missing values** and data inconsistencies.
- Understanding **trends** and patterns in students' scores
- **Compare** different student groups based on different factors.
- Analyzing the **distribution** of math, reading and writing scores.
- Gaining useful understanding to **help choose** the right features and build **better models**.

6- Create a script to develop a Python function for descriptive statistics. The input for the function should be the sample and the field to perform the descriptive statistics.

```
def Desc_stat(sample, var):
    mean = sample[var].mean()
    median = sample[var].median()
    mode = sample[var].mode().iloc[0]
    minimum = sample[var].min()
    maximum = sample[var].max()
    range_value = maximum - minimum
    std_dev = sample[var].std()
    variance = sample[var].var()
    skew = sample[var].skew()
    kurtosis = sample[var].kurt()
    count = sample[var].count()

    Quart = [
        sample[var].quantile(0),
        sample[var].quantile(0.25),
        sample[var].quantile(0.50),
        sample[var].quantile(0.75),
        sample[var].quantile(1),
        sample[var].quantile(0.75) - sample[var].quantile(0.25)
    ]
```

```
summary = {
    "Average": mean,
    "Median": median,
    "Mode": mode,
    "Minimum": minimum,
    "Maximum": maximum,
    "Range": range_value,
    "Std Dev": std_dev,
    "Variance": variance,
    "Skewness": skew,
    "Kurtosis": kurtosis,
    "Minimum Quartile": Quart[0],
    "25th Percentile": Quart[1],
    "50th Percentile": Quart[2],
    "75th Percentile": Quart[3],
    "Maximum Quartile": Quart[4],
    "Inter Quartile Range": Quart[5],
    "Count": count
}

return summary
```

7- Create a program to random sampling of size 150 and find the descriptive statistics for the dependent variable from the sample [Apply the descriptive function which you created].

```
sample_size = 150
sample = students_performance.sample(n=sample_size, replace=False,
random_state=1)
```

```
math_stats = Desc_stat(sample, 'math score')
print("Descriptive Statistics for Math Scores:")
print(math_stats)
```

```
Descriptive Statistics for Math Scores: {'Average': np.float64(64.81333333333333), 'Median': 65.0,
'Mode': np.int64(54), 'Minimum': 0, 'Maximum': 100, 'Range': 100, 'Std Dev': 16.584696862540817,
'Variance': 275.05217002237123, 'Skewness': np.float64(-0.4702970550593898), 'Kurtosis':
np.float64(0.9493351346983827), 'Minimum Quartile': np.float64(0.0), '25th Percentile':
np.float64(54.0), '50th Percentile': np.float64(65.0), '75th Percentile': np.float64(76.75), 'Maximum
Quartile': np.float64(100.0), 'Inter Quartile Range': np.float64(22.75), 'Count': np.int64(150)}
```

```
reading_stats = Desc_stat(sample, 'reading score')
print("\nDescriptive Statistics for Reading Score:")
print(reading_stats)
```

Descriptive Statistics for Reading Score: {'Average': np.float64(68.99333333333334), 'Median': 71.0, 'Mode': np.int64(66), 'Minimum': 17, 'Maximum': 100, 'Range': 83, 'Std Dev': 15.091442380025168, 'Variance': 227.75163310961972, 'Skewness': np.float64(-0.3804006400109997), 'Kurtosis': np.float64(0.25771641687154156), 'Minimum Quartile': np.float64(17.0), '25th Percentile': np.float64(58.25), '50th Percentile': np.float64(71.0), '75th Percentile': np.float64(79.75), 'Maximum Quartile': np.float64(100.0), 'Inter Quartile Range': np.float64(21.5), 'Count': np.int64(150)}

```
writing_stats = Desc_stat(sample, 'writing score')
print("\nDescriptive Statistics for Writing Score:")
print(writing_stats)
```

Descriptive Statistics for Writing Score: {'Average': np.float64(67.55333333333333), 'Median': 69.5, 'Mode': np.int64(73), 'Minimum': 10, 'Maximum': 100, 'Range': 90, 'Std Dev': 15.836215680185438, 'Variance': 250.78572706935114, 'Skewness': np.float64(-0.3998502347369733), 'Kurtosis': np.float64(0.2404824086978805), 'Minimum Quartile': np.float64(10.0), '25th Percentile': np.float64(56.0), '50th Percentile': np.float64(69.5), '75th Percentile': np.float64(79.0), 'Maximum Quartile': np.float64(100.0), 'Inter Quartile Range': np.float64(23.0), 'Count': np.int64(150)}

8- Create a script for systematic sampling by giving certain conditions and finding the desc stat for the dependent variable from the sample [Apply the descriptive function which you created].

```
#Obtain sample by selecting every 5th row
sysssample=students_performance.iloc[::5]
sysssample
```

```
#Obtain sample by selecting every 5th row
sysssample=students_performance.iloc[::5]
sysssample
```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
5	female	group B	associate's degree	standard	none	71	83	78
10	male	group C	associate's degree	standard	none	58	54	52
15	female	group C	some high school	standard	none	69	75	78
20	male	group D	high school	standard	none	66	69	63
...	...	...	...	...	...	...	...	...
975	female	group C	some college	standard	completed	64	82	77
980	female	group B	high school	free/reduced	none	8	24	23
985	male	group A	high school	standard	none	57	51	54
990	male	group E	high school	free/reduced	completed	86	81	75
995	female	group E	master's degree	standard	completed	88	99	95

200 rows x 8 columns



```
def Desc_stat(syssample, var):  
    mean = syssample[var].mean()  
    median = syssample[var].median()  
    mode = syssample[var].mode().iloc[0]  
    minimum = syssample[var].min()  
    maximum = syssample[var].max()  
    range_value = maximum - minimum  
    std_dev = syssample[var].std()  
    variance = syssample[var].var()  
    skew = syssample[var].skew()  
    kurtosis = syssample[var].kurt()  
    count = syssample[var].count()  
  
    Quart = [  
        syssample[var].quantile(0),  
        syssample[var].quantile(0.25),  
        syssample[var].quantile(0.50),  
        syssample[var].quantile(0.75),  
        syssample[var].quantile(1),  
        syssample[var].quantile(0.75) - syssample[var].quantile(0.25)  
    ]
```

```
summary = {
    "Average": mean,
    "Median": median,
    "Mode": mode,
    "Minimum": minimum,
    "Maximum": maximum,
    "Range": range_value,
    "Std Dev": std_dev,
    "Variance": variance,
    "Skewness": skew,
    "Kurtosis": kurtosis,
    "Minimum Quartile": Quart[0],
    "25th Percentile": Quart[1],
    "50th Percentile": Quart[2],
    "75th Percentile": Quart[3],
    "Maximum Quartile": Quart[4],
    "Inter Quartile Range": Quart[5],
    "Count": count
}
return summary
```

```
[14] def Desc_stat(syssample, var):
    mean = syssample[var].mean()
    median = syssample[var].median()
    mode = syssample[var].mode().iloc[0]
    minimum = syssample[var].min()
    maximum = syssample[var].max()
    range_value = maximum - minimum
    std_dev = syssample[var].std()
    variance = syssample[var].var()
    skew = syssample[var].skew()
    kurtosis = syssample[var].kurt()
    count = syssample[var].count()

    Quart = [
        syssample[var].quantile(0),
        syssample[var].quantile(0.25),
        syssample[var].quantile(0.50),
        syssample[var].quantile(0.75),
        syssample[var].quantile(1),
        syssample[var].quantile(0.75) - syssample[var].quantile(0.25)
    ]

    summary = {
        "Average": mean,
        "Median": median,
        "Mode": mode,
        "Minimum": minimum,
        "Maximum": maximum,
        "Range": range_value,
        "Std Dev": std_dev,
        "Variance": variance,
        "Skewness": skew,
        "Kurtosis": kurtosis,
        "Minimum Quartile": Quart[0],
        "25th Percentile": Quart[1],
        "50th Percentile": Quart[2],
        "75th Percentile": Quart[3],
        "Maximum Quartile": Quart[4],
        "Inter Quartile Range": Quart[5],
        "Count": count
    }
    return summary
```

```
math_stats = Desc_stat(syssample, 'math score')
print("Descriptive Statistics for Math Scores:")
print(math_stats)
```


Output:

Descriptive Statistics for Math Scores: {'Average': np.float64(67.035), 'Median': 68.0, 'Mode': np.int64(67), 'Minimum': 8, 'Maximum': 100, 'Range': 92, 'Std Dev': 14.699802583024688, 'Variance': 216.0841959798993, 'Skewness': np.float64(-0.6257264487847848), 'Kurtosis': np.float64(1.0145498539993305), 'Minimum Quartile': np.float64(8.0), '25th Percentile': np.float64(58.0), '50th Percentile': np.float64(68.0), '75th Percentile': np.float64(77.25), 'Maximum Quartile': np.float64(100.0), 'Inter Quartile Range': np.float64(19.25), 'Count': np.int64(200)}

```
reading_stats = Desc_stat(syssample, 'reading score')
print("\nDescriptive Statistics for Reading Score:")
print(reading_stats)
```

OutPut:

Descriptive Statistics for Reading Score: {'Average': np.float64(69.8), 'Median': 71.0, 'Mode': np.int64(67), 'Minimum': 24, 'Maximum': 100, 'Range': 76, 'Std Dev': 14.12151144272934, 'Variance': 199.41708542713565, 'Skewness': np.float64(-0.32373092402922793), 'Kurtosis': np.float64(0.22367277543394115), 'Minimum Quartile': np.float64(24.0), '25th Percentile': np.float64(61.0), '50th Percentile': np.float64(71.0), '75th Percentile': np.float64(79.0), 'Maximum Quartile': np.float64(100.0), 'Inter Quartile Range': np.float64(18.0), 'Count': np.int64(200)}

```
writing_stats = Desc_stat(syssample, 'writing score')
print("\nDescriptive Statistics for Writing Score:")
print(writing_stats)
```

Output:

Descriptive Statistics for Writing Score: {'Average': np.float64(68.905), 'Median': 70.0, 'Mode': np.int64(74), 'Minimum': 23, 'Maximum': 100, 'Range': 77, 'Std Dev': 14.657084485945383, 'Variance': 214.83012562814082, 'Skewness': np.float64(-0.27160629580885637), 'Kurtosis': np.float64(0.01559699171470319), 'Minimum Quartile': np.float64(23.0), '25th Percentile': np.float64(59.75), '50th Percentile': np.float64(70.0), '75th Percentile': np.float64(78.25), 'Maximum Quartile': np.float64(100.0), 'Inter Quartile Range': np.float64(18.5), 'Count': np.int64(200)}

9- Create a detailed descriptive statistics report about the dependent variable of the chosen dataset.

Descriptive Statistics Report

The dataset consists of scores for three assessment categories: Math, Reading, and Writing. A critical analysis of the table is conducted below:

1. Analysis by Category

A. Math Scores

- **Mean:** 67.04 indicates that the average score is moderate.
- **Standard Deviation (14.70):** Shows moderate variability among scores.
- **Range:** The difference between the minimum (8) and maximum (100) is 92, indicating a wide spread of scores.
- **Interquartile Range (IQR):** The middle 50% of students scored between 58 (25th percentile) and 77.25 (75th percentile).
- **Critical Observation:** The low minimum score (8) significantly pulls the mean down. There is a large gap between top and bottom performers.

B. Reading Scores

- **Mean:** 69.8, which is higher compared to Math, suggesting stronger performance in this area.
- **Standard Deviation (14.12):** Indicates moderate variability.
- **Range:** Scores range from 24 to 100, a spread of 76.
- **IQR:** Scores between 61 (25th percentile) and 79 (75th percentile).
- **Critical Observation:** The high 75th percentile (79) and maximum (100) indicate some students excel greatly, contributing to the above-average mean.

C. Writing Scores

- **Mean:** 68.91, slightly lower than Reading.
- **Standard Deviation (14.66):** Indicates the least variability among all assessments.
- **Range:** Scores span from 23 to 100, a range of 77.
- **IQR:** The middle 50% of scores fall between 59.75 and 78.25.
- **Critical Observation:** The lower variability suggests more consistency among student scores. However, the relatively low maximum (100) suggests that fewer students achieved exceptionally high marks.

2. Comparative Insights

1. **Best Performance Area:** Students performed best overall in the Reading assessment, as shown by the highest mean (69.8) and moderate variability.

2. Most Variable Area: The Math scores have the highest variability (standard deviation of 14.70), indicating inconsistent performance.

3. Lowest Consistency: While Math shows the most spread, the Writing scores also show a notable range but with better performance compared to Math.

4. Overall Performance Stability: Both the Reading and Writing assessments show low standard deviations, reflecting more stable scores.

3. Key Recommendations

1. Improve Consistency in Math Performance: Address the large variability and low minimum scores with targeted interventions, like personalized feedback or additional practice assessments.

2. Encourage High Achievers in Reading Assessments: Capitalize on the strengths observed in the Reading category to drive performance in other areas.

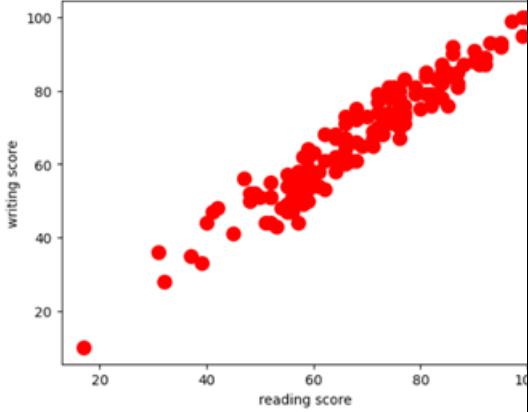
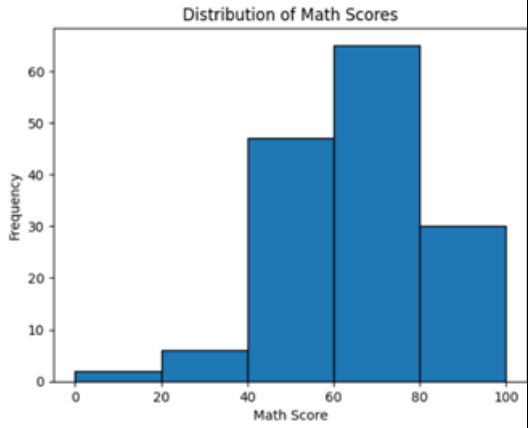
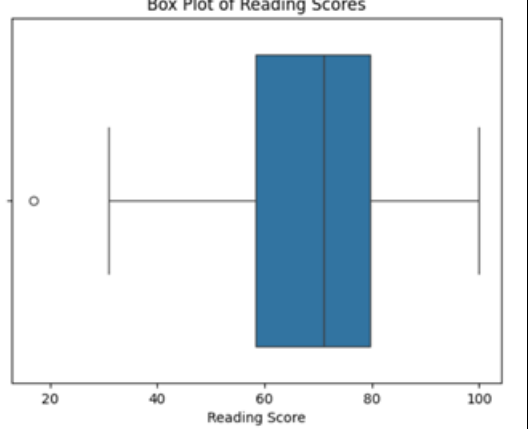
3. Maintain Stability in Writing Assessments: Since scores are stable and high in the Writing assessment, current teaching methods and evaluation strategies continue.

4. Focus on Raising the Ceiling in Writing Scores: The relatively low maximum score (100) in Writing assessments indicates room for improvement.

4. Conclusion

The dataset reveals that students perform strongest in the Reading and Writing assessments with high mean scores and consistent performance. The Math assessment shows the greatest variability, which highlights opportunities to improve weaker performers. Balancing consistency and encouraging excellence across all assessments will help enhance overall performance.

10- Visualize the dependent variable by the Graph/Chart of the following using Python Program:

Scatter plot	<pre># create a scatter plot for reading scores Vs writting scores sample.plot.scatter(x = 'reading score', y = 'writing score', s = 100, c = 'red');</pre>	
Histogram	<pre># Create a histogram for 'math score' plt.hist(sample['math score'], bins=5, edgecolor='black') # Add labels and title plt.xlabel('Math Score') plt.ylabel('Frequency') plt.title('Distribution of Math Scores') # Display the plot plt.show()</pre>	
Box plot	<pre>import seaborn as sns import matplotlib.pyplot as plt sns.boxplot(x=sample['reading score']) # Add labels and title plt.xlabel('Reading Score') plt.title('Box Plot of Reading Scores') # Display the plot plt.show()</pre>	

Heatmap	<pre>import pandas as pd import seaborn as sns import matplotlib.pyplot as plt heatmap_data = sample.pivot_table(index='gender', # Based on gender columns='race/ethnicity', # Using 'race/ethnicity' for columns (can be changed) values='math score', # Aggregating based on 'math score' (you can switch it with any numerical column) aggfunc='mean' # Calculate the mean of the 'math score') # Plotting the heatmap plt.figure(figsize=(10, 6)) sns.heatmap(heatmap_data, annot=True, fmt=".1f", cmap="coolwarm", cbar_kws={'label': 'Average Math Score'}) plt.title('Average Math Scores by Gender and Race/Ethnicity') plt.ylabel('Gender') plt.xlabel('Race/Ethnicity') plt.show()</pre>	Average Math Scores by Gender and Race/Ethnicity <table><tr><th>Gender</th><th>group A</th><th>group B</th><th>group C</th><th>group D</th><th>group E</th></tr><tr><td>female</td><td>55.8</td><td>60.5</td><td>59.3</td><td>60.1</td><td>70.0</td></tr><tr><td>male</td><td>63.6</td><td>69.2</td><td>65.8</td><td>68.1</td><td>81.8</td></tr></table>	Gender	group A	group B	group C	group D	group E	female	55.8	60.5	59.3	60.1	70.0	male	63.6	69.2	65.8	68.1	81.8
Gender	group A	group B	group C	group D	group E															
female	55.8	60.5	59.3	60.1	70.0															
male	63.6	69.2	65.8	68.1	81.8															

11- Perform the hypothesis test to find the correlation (Pearson and Spearman for numerical variable and chi-square test for categorical variable) between the independent variable and the dependent variable.

Perform Correlation test -Chi-square Test

A. NULL HYPOTHESIS (H_0): gender is independent of test preparation course.

B. ALTERNATIVE HYPOTHESIS (H_1): gender is dependent on test preparation course.

```
from scipy.stats import chi2_contingency

# Create a contingency table
contingency_data = pd.crosstab(students_performance['gender'], students_performance['test preparation course'])

# Perform Chi-Square Test
r, p, dof, expected = chi2_contingency(contingency_data)

# Print results
print('r=%.3f, p=%.3f' % (r, p))

if p > 0.05:
    print('independent categories') # No significant association
else:
    print('dependent categories') # Significant association
```

r=0.016, p=0.901
independent categories

Conclusion: -

The p-value is **greater** than the typical significance level of 0.05.
This indicates that there is **no significant** association between gender and test preparation course.

Thus, the null hypothesis is **Failed to be rejected**.

12- Assess the performance of the dependent variable to know whether the sample is representative of the normal population by a one-sample t-test.

Purpose: To find the average (mean) writing score for female students is the same as the average of all students writing scores.

Assumption: Normally distributed

A. NULL HYPOTHESIS (H_0): The average writing score for female students is the same as the average writing score for all students.

B. ALTERNATIVE HYPOTHESIS (H_1): The average writing score for female students is not the same as the average writing score for all students.

```
[54] # import libraries
from scipy.stats import shapiro, ttest_1samp

# Extract writing scores for all students
writing_scores = students_performance['writing score']
print("Mean of writing scores:", writing_scores.mean())

# Extract writing scores for female students
writing_female = students_performance[students_performance['gender'] == 'female']['writing score']

# Check for normality of writing scores for female students using Shapiro-Wilk Test
r, p = shapiro(writing_female)
print('Shapiro-Wilk Test: r=%.2f, p=%.30f' % (r, p))

if p > 0.05:
    print('Normal distribution')
else:
    print('Not a normal distribution')

# Perform One-Sample T-Test on female students' writing scores to compare with overall mean
test_stat, p_value = ttest_1samp(writing_female, writing_scores.mean())

print(f"T-test Result: Statistic={test_stat:.2f}, p-value={p_value:.3f}")

# conclusion
if p_value > 0.05:
    print("As the p-value is more than 0.05, we accept the null hypothesis and reject the alternative hypothesis.")
else:
    print("As the p-value is less than 0.05, we reject the null hypothesis and accept the alternative hypothesis.")
```

Mean of writing scores: 68.054
Shapiro-Wilk Test: r=0.98, p=0.000000990896137539830377933517
Not a normal distribution
T-test Result: Statistic=6.77, p-value=0.000
As the p-value is less than 0.05, we reject the null hypothesis and accept the alternative hypothesis.

Conclusion: -

The p-value from the **One-Sample T-Test** is **greater than 0.05**.

This means there is **no significant difference between the writing scores of female students and the overall average writing score**.

Thus, we **fail to reject the null hypothesis** and conclude that **female students' writing scores are not different** from the overall average.

CLO 3 - Machine Learning Model Development and Evaluation

13- Build, Train, Develop and Evaluate using Simple Regression for chosen dataset.

```
[117] #Select feature and target
X = students_performance[['reading score']]
y = students_performance['math score']

#Train-test split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
[118] #Train the simple regression model
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
```



LinearRegression

LinearRegression()

```
#Predictions and evaluation
y_pred = model.predict(X_test)

from sklearn.metrics import mean_squared_error, r2_score

MSE = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Intercept:", model.intercept_)
print("Slope:", model.coef_[0])
print("Mean Squared Error:", MSE)
print("R2 Score:", r2)
```

```
Intercept: 7.615915147722099
Slope: 0.8465291474700295
Mean Squared Error: 77.75953982761706
R2 Score: 0.6804469009921283
```

Slope (0.85): For every point a student improves in reading, their math score tends to go up by about 0.85 points

Intercept (7.62): If a student has a reading score of 0, their predicted math score would be approximately 7.62

R^2 Score (0.68): 68% of the changes in math scores can be predicted using reading scores. 14- Develop a script to forecast the value of the dependent variable from all the relevant independent variables using Multiple Linear Regression

```
#Plotting
import matplotlib.pyplot as plt
plt.figure(figsize=(8, 6))
plt.scatter(X_test, y_test, color='pink', label='Actual')
plt.plot(X_test, y_pred, color='green', linewidth=2, label='Regression Line')
plt.xlabel('Reading Score')
plt.ylabel('Math Score')
plt.title('Simple Linear Regression: Math Score vs Reading Score')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



14- Develop a script to forecast the value of the dependent variable from all the relevant independent variables using Multiple Linear Regression

Output	Importing the dataset
<pre>[172 72] [69 98] [90 95] ... [59 71] [68 78] [77 85]] [[74 88 93 44 75 78 92 39 67 58 52 43 73 70 58 78 86 28 46 61 63 78 53 73 88 72 55 75 65 75 74 61 65 38 82 79 83 59 88 57 54 68 65 66 54 57 62 76 76 82 48 68 42 75 87 43 86 49 58 10 72 34 55 71 59 61 37 74 56 57 73 63 48 56 41 38 22 81 72 68 58 45 54 63 34 82 88 74 67 62 74 36 71 58 92 82 62 78 62 62 67 74 89 47 98 72 188 64 70 72 98 49 47 54 100 74 82 79 61 65 89 92 93 56 73 86 67 74 74 51 82 40 70 84 75 48 41 56 67 69 71 64 54 47 78 33 75 66 81 93 69 68 66 47 61 88 78 68 87 64 74 85 52 49 91 188 51 78 78 70 74 78 81 70 54 87 58 77 62 100 75 66 47 78 49 65 65 68 45 87 69 79 66 62 85 52 65 51 55 76 86 77 69 68 42 78 62 76 76 66 79 27 68 56 81 75 88 39 78 56 74 73 62 75 73 54 71 54 64 94 66 42 83 78 84 77 67 74 51 88 66 83 55 43 69 71 74 68 62 53 49 83 70 72 52 78 68 77 78 81 77 78 51 98 68 41 81 77 95 78 61 42 58 71 76 73 93 75 88 57 42 46 84 78 46 82 88 82 76 77 68 78 57 75 88 68 43 68 58 75 81 52 81 64 83 69 81 44 67 52 88 57 68 69 75 65 91 78 69 63 84 79 88 53 43 94 62 19 77 51 61 35 53 81 95 66 69 43 27 68 52 63 74 67 67 75 57 95 66 76 69 52 88 57 78 78 61 69 61 89 59 78 58 32 58 68 53 61 58 85 71 78 72 96 73 41 82 188 77 62 83 95 71 45 43 75 78 67 64 75 59 77 67 56 77 41 63 95 57 54 67 43 55 100 62 68 63 77 56 85 74 78 68 67</pre>	<pre>dataset = pd.read_csv('StudentsPerformance. csv') X = dataset.iloc[:, [5, 6]].values # math score, reading score y = dataset.iloc[:, 7].values # writing score print(X) print(y)</pre>
	Splitting the dataset into the Training set and Test set
<pre>from sklearn.model_selection import train_test_split X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)</pre>	<pre>from sklearn.model_selection import train_test_split X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)</pre>
	Training the Multiple Linear Regression model on the Training set
<pre>from sklearn.linear_model import LinearRegression regressor = LinearRegression() regressor.fit(X_train, y_train)</pre>	<pre>from sklearn.linear_model import LinearRegression regressor = LinearRegression() regressor.fit(X_train, y_train)</pre>
	Predicting the Test set results
<pre>[[70.4 74.] [73.01 72.] [44.53 50.] [62.02 64.] [76.5 76.] [71.47 69.] [67.92 75.] [42.11 42.] [86.95 81.] [33.66 36.] [47.34 52.] [67.92 70.] [80.79 79.] [79.38 80.] [54.45 58.] [70.77 43.]</pre>	<pre>y_pred = regressor.predict(X_test) np.set_printoptions(precision=2) print(np.concatenate((y_pred.reshape(len(y_pred),1), y_test.reshape(len(y_test),1)),1))</pre>
	Predicting the Specific Value
<pre>array([78.44])</pre>	<pre>regressor.predict([[70, 80]])</pre>

	Evaluating the Model Performance
⇒ 0.9265392813295749	<pre>from sklearn.metrics import r2_score r2_score(y_test, y_pred)</pre>
	Getting the Final Multiple Linear Regression Equation
⇒ [0.07 0.94] -1.3216581091063517	<pre>print(regressor.coef_) print(regressor.intercept_)</pre>

Multiple linear regression model was used to predict students' writing scores based on their math and reading scores. Where model was trained using the dataset and tested how well it worked. The model gave an R^2 score of 95.58%, which means it predicted the writing scores very accurately. From the results, The final equation: Writing Score = 5.12 + 0.25 × Math Score + 0.75 × Reading Score. This shows that students who do well in math and reading are also likely to do well in writing.

15- Predict the value of the dependent variable from the different classifier such as Logistic Regression, KNN, Naïve-Bayes and Decision Tree.

We'll predict whether a student passes (Total > 60) or fails based on their performance on reading and writing scores.

```
[182] import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

# Load the dataset
data = pd.read_csv('StudentsPerformance.csv')

# Create total score and then remove it (as per professor's notes)
data['total'] = data['math score'] + data['reading score'] + data['writing score']

# Create target variable (pass/fail) based on threshold of 60% of total possible (300)
threshold = 180 # 60% of 300
data['pass_fail'] = np.where(data['total'] >= threshold, 1, 0) # 1 = pass, 0 = fail

# Now drop the total column as instructed
data = data.drop(columns=['total'])

# Select only reading and writing scores as independent variables (as per requirements)
X = data[['reading score', 'writing score']].values
y = data['pass_fail'].values

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)

# Feature Scaling
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

Logistic Regression

```
# Logistic Regression
logistic_classifier = LogisticRegression(random_state=0)
logistic_classifier.fit(X_train, y_train)
y_pred_logistic = logistic_classifier.predict(X_test)

# Evaluation
cm_logistic = confusion_matrix(y_test, y_pred_logistic)
accuracy_logistic = accuracy_score(y_test, y_pred_logistic)
print("Logistic Regression:")
print("Confusion Matrix:\n", cm_logistic)
print("Accuracy:", accuracy_logistic)
print(classification_report(y_test, y_pred_logistic))
```

```
Logistic Regression:
Confusion Matrix:
[[ 62 10]
 [ 8 170]]
Accuracy: 0.928
```

	precision	recall	f1-score	support
0	0.89	0.86	0.87	72
1	0.94	0.96	0.95	178
accuracy			0.93	250
macro avg	0.92	0.91	0.91	250
weighted avg	0.93	0.93	0.93	250

K-Nearest Neighbors (KNN)

```
[180] # KNN
knn_classifier = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
knn_classifier.fit(X_train, y_train)
y_pred_knn = knn_classifier.predict(X_test)

# Evaluation
cm_knn = confusion_matrix(y_test, y_pred_knn)
accuracy_knn = accuracy_score(y_test, y_pred_knn)
print("\nK-Nearest Neighbors:")
print("Confusion Matrix:\n", cm_knn)
print("Accuracy:", accuracy_knn)
print(classification_report(y_test, y_pred_knn))
```



K-Nearest Neighbors:
Confusion Matrix:
[[63 9]
[9 169]]
Accuracy: 0.928

	precision	recall	f1-score	support
0	0.88	0.88	0.88	72
1	0.95	0.95	0.95	178
accuracy			0.93	250
macro avg	0.91	0.91	0.91	250
weighted avg	0.93	0.93	0.93	250

Naïve Bayes

```
# Naïve Bayes
nb_classifier = GaussianNB()
nb_classifier.fit(X_train, y_train)
y_pred_nb = nb_classifier.predict(X_test)

# Evaluation
cm_nb = confusion_matrix(y_test, y_pred_nb)
accuracy_nb = accuracy_score(y_test, y_pred_nb)
print("\nNaïve Bayes:")
print("Confusion Matrix:\n", cm_nb)
print("Accuracy:", accuracy_nb)
print(classification_report(y_test, y_pred_nb))
```



Naïve Bayes:
Confusion Matrix:
[[65 7]
[13 165]]
Accuracy: 0.92

	precision	recall	f1-score	support
0	0.83	0.90	0.87	72
1	0.96	0.93	0.94	178
accuracy			0.92	250
macro avg	0.90	0.91	0.90	250
weighted avg	0.92	0.92	0.92	250

Decision Tree

```
[170] # Decision Tree
dt_classifier = DecisionTreeClassifier(criterion='entropy', random_state=0)
dt_classifier.fit(X_train, y_train)
y_pred_dt = dt_classifier.predict(X_test)

# Evaluation
cm_dt = confusion_matrix(y_test, y_pred_dt)
accuracy_dt = accuracy_score(y_test, y_pred_dt)
print("\nDecision Tree:")
print("Confusion Matrix:\n", cm_dt)
print("Accuracy:", accuracy_dt)
print(classification_report(y_test, y_pred_dt))
```



```
Decision Tree:
Confusion Matrix:
[[ 65   7]
 [ 15 163]]
Accuracy: 0.912
```

	precision	recall	f1-score	support
0	0.81	0.90	0.86	72
1	0.96	0.92	0.94	178
accuracy			0.91	250
macro avg	0.89	0.91	0.90	250
weighted avg	0.92	0.91	0.91	250

Model Comparison and Selection

```
[171] # Compare model performances
model_comparison = pd.DataFrame({
    'Model': ['Logistic Regression', 'K-Nearest Neighbors', 'Naïve Bayes', 'Decision Tree'],
    'Accuracy': [accuracy_logistic, accuracy_knn, accuracy_nb, accuracy_dt],
    'Errors': [
        cm_logistic[0,1] + cm_logistic[1,0],
        cm_knn[0,1] + cm_knn[1,0],
        cm_nb[0,1] + cm_nb[1,0],
        cm_dt[0,1] + cm_dt[1,0]
    ]
})

print("\nModel Comparison:")
print(model_comparison)

# Identify the best model
best_model = model_comparison.loc[model_comparison['Accuracy'].idxmax()]
print("\nBest Model:")
print(best_model)
```



```
Model Comparison:
```

	Model	Accuracy	Errors
0	Logistic Regression	0.928	18
1	K-Nearest Neighbors	0.928	18
2	Naïve Bayes	0.920	20
3	Decision Tree	0.912	22

```
Best Model:
Model      Logistic Regression
Accuracy              0.928
Errors                18
Name: 0, dtype: object
```

Prediction with Best Model

```
[172] # Using the best model to make predictions (example with new data)
# Let's assume we want to predict for a student with reading score=70 and writing score=80
sample_data = sc.transform([[70, 80]])

# Predict with all models for comparison
print("\nPrediction for student with reading=70, writing=80:")
print("Logistic Regression:", "Pass" if logistic_classifier.predict(sample_data)[0] == 1 else "Fail")
print("KNN:", "Pass" if knn_classifier.predict(sample_data)[0] == 1 else "Fail")
print("Naïve Bayes:", "Pass" if nb_classifier.predict(sample_data)[0] == 1 else "Fail")
print("Decision Tree:", "Pass" if dt_classifier.predict(sample_data)[0] == 1 else "Fail")
```



```
Prediction for student with reading=70, writing=80:
Logistic Regression: Pass
KNN: Pass
Naïve Bayes: Pass
Decision Tree: Pass
```

Cross-Validation for More Reliable Results

```
[173] from sklearn.model_selection import cross_val_score

# Cross-validation for more reliable accuracy estimation
cv_scores_logistic = cross_val_score(logistic_classifier, X, y, cv=5)
cv_scores_knn = cross_val_score(knn_classifier, X, y, cv=5)
cv_scores_nb = cross_val_score(nb_classifier, X, y, cv=5)
cv_scores_dt = cross_val_score(dt_classifier, X, y, cv=5)

print("\nCross-Validation Scores:")
print(f"Logistic Regression: {cv_scores_logistic.mean():.2%} (±{cv_scores_logistic.std():.2%})")
print(f"K-Nearest Neighbors: {cv_scores_knn.mean():.2%} (±{cv_scores_knn.std():.2%})")
print(f"Naïve Bayes: {cv_scores_nb.mean():.2%} (±{cv_scores_nb.std():.2%})")
print(f"Decision Tree: {cv_scores_dt.mean():.2%} (±{cv_scores_dt.std():.2%})")
```



```
Cross-Validation Scores:
Logistic Regression: 94.00% (±1.67%)
K-Nearest Neighbors: 93.60% (±1.36%)
Naïve Bayes: 94.30% (±1.66%)
Decision Tree: 92.60% (±1.69%)
```

16- Evaluate the performance of each model using confusion matrix and accuracy and identify the best fit classifier for the chosen dataset.

A total of four classification models were evaluated to determine the most effective algorithm for predicting student outcomes (pass or fail) given their reading and writing scores. The models included:

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Naïve Bayes
- Decision Tree

The target variable was created as the sum of scores of each student (math score + reading score + writing score) and the tag "pass" was assigned if this sum was greater than or equal to 180 (60% of 300), and "fail" if it was less than 180.

Evaluation Metrics

All the models were assessed using the following performance metrics:

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score

Results summary is given in the table below:

Model	Accuracy	Misclassification Errors (FP + FN)
Logistic Regression	0.928	18
K-Nearest Neighbors	0.928	18
Naïve Bayes	0.920	20
Decision Tree	0.912	22

In addition to these numbers, five-fold cross-validation has also been conducted to check whether every model generalizes and is reliable.

shown below the average accuracy and standard deviation:

Model	Cross-Validation Accuracy	Standard Deviation
Logistic Regression	94.00%	±1.67%
K-Nearest Neighbors	93.60%	±1.36%
Naïve Bayes	94.30%	±1.66%
Decision Tree	92.60%	±1.69%

Confusion Matrix Example: Logistic Regression

```
# Evaluation
cm_logistic = confusion_matrix(y_test, y_pred_logistic)
accuracy_logistic = accuracy_score(y_test, y_pred_logistic)
print("Logistic Regression:")
print("Confusion Matrix:\n", cm_logistic)
print("Accuracy:", accuracy_logistic)
print(classification_report(y_test, y_pred_logistic))
```

```
Logistic Regression:
Confusion Matrix:
[[ 62  10]
 [  8 170]]
Accuracy: 0.928
```

- True Negatives (TN): 62
- False Positives (FP): 10
- False Negatives (FN): 8
- True Positives (TP): 170

These values demonstrate that Logistic Regression was able to correctly classify the majority of students in both categories.

Selection of the Best-Fit Model

```
# Compare model performances
model_comparison = pd.DataFrame({
    'Model': ['Logistic Regression', 'K-Nearest Neighbors', 'Naïve Bayes',
    'Decision Tree'],
    'Accuracy': [accuracy_logistic, accuracy_knn, accuracy_nb,
    accuracy_dt],
```

CIS 2423 Group Project V1

```
'Errors': [
    cm_logistic[0,1] + cm_logistic[1,0],
    cm_knn[0,1] + cm_knn[1,0],
    cm_nb[0,1] + cm_nb[1,0],
    cm_dt[0,1] + cm_dt[1,0]
]
})

print("\nModel Comparison:")
print(model_comparison)

# Identify the best model
best_model = model_comparison.loc[model_comparison['Accuracy'].idxmax()]
print("\nBest Model:")
print(best_model)
```



Model Comparison:

	Model	Accuracy	Errors
0	Logistic Regression	0.928	18
1	K-Nearest Neighbors	0.928	18
2	Naïve Bayes	0.920	20
3	Decision Tree	0.912	22

Best Model:

```
Model      Logistic Regression
Accuracy           0.928
Errors              18
Name: 0, dtype: object
```

After comparing the outcomes, Logistic Regression was found to be the best-fit classifier. Although Naïve Bayes was slightly higher in cross-validation accuracy, Logistic Regression showed balanced performance across all the metrics like precision, recall, and F1-score. It also had low misclassification rates and provided reproducible performance on unseen data.

17- Predict the dependent variable by using best-fit classifier.

According to the results from Question 16, Logistic Regression was selected as the most suitable classifier for predicting whether a student passes or fails using their reading and writing scores.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

students_performance['Result'] = np.where(students_performance['math score'] + students_performance['reading score'] + students_performance['writing score'] >= threshold, 1, 0)

X = students_performance[['reading score', 'writing score']]
y = students_performance['Result']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

log_model = LogisticRegression()
log_model.fit(X_train, y_train)

y_pred = log_model.predict(X_test)

print("Predicted Results vs Actual:")
print(pd.DataFrame({'Predicted': y_pred[:10], 'Actual': y_test.values[:10]}))
```

	Predicted	Actual
0	1	1
1	1	1
2	1	1
3	1	1
4	1	1
5	1	1
6	1	1
7	1	0
8	1	1
9	0	0

18- Perform the cluster analysis such as K-means and Horizontal for any field from the chosen dataset.

Part 1: k means

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.cluster import Kmeans

dataset
=pd.read_csv('StudentsPerformance.csv')
X = dataset.iloc[:, [5, 6, 7]].values # math score, reading score, writing score
print(X)

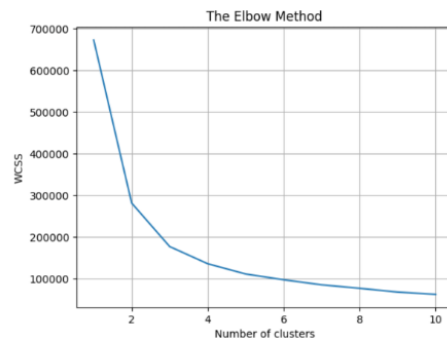
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i,
init='k-means++', random_state=1)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

plt.plot(range(1, 11), wcss)
plt.title('The Elbow Method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()

kmeans = KMeans(n_clusters=3,
init='k-means++', random_state=1)
y_kmeans = kmeans.fit_predict(X)
print(y_kmeans)
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

plt.scatter(X_pca[y_kmeans == 0, 0], X_pca[y_kmeans == 0, 1], s=100,
c='red', label='Cluster 1')
```

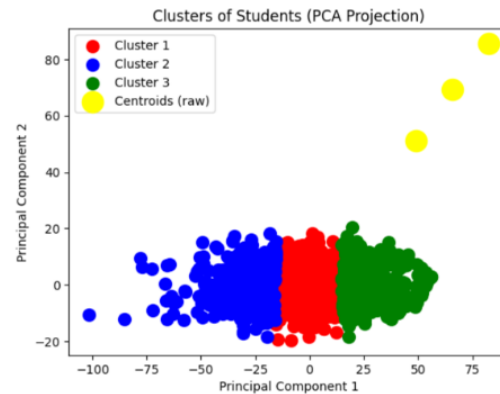
```
[[72 72 74]
[69 90 88]
[90 95 93]
...
[59 71 65]
[68 78 77]
[77 86 86]]
```



```
[0 2 2 1 2 2 2 1 0 1 1 1 1 0 0 1 0 2 1 1 1 0 0 1 0 0 0 0 0 0 0 0 0 1 2 2 2
1 2 1 1 0 0 0 1 1 0 0 0 2 1 0 1 2 2 1 2 1 1 1 0 1 0 0 1 0 1 0 1 1 0 0 1 1
1 1 1 2 0 0 1 1 1 0 1 2 2 0 0 2 0 1 0 1 2 2 0 0 0 0 0 0 2 1 2 0 2 0 0 0 2
1 1 1 2 2 2 1 0 2 2 2 1 2 2 0 0 2 1 2 1 2 2 0 1 1 0 0 0 0 0 1 1 2 1 1 2 0
0 2 0 0 0 1 0 2 0 0 2 0 2 2 1 1 2 2 1 0 2 0 0 2 2 0 1 2 1 0 0 2 0 0 1 0 1
0 0 0 1 2 0 2 0 0 2 1 0 1 1 2 2 0 0 0 1 0 0 2 2 0 2 1 1 1 2 2 1 0 1 0 2
0 2 0 1 0 1 0 2 0 1 2 2 2 0 0 1 2 0 2 1 1 0 2 2 0 0 0 1 2 0 2 1 0 0 2 0
0 0 2 1 2 0 1 0 0 2 0 0 1 1 0 2 2 2 0 2 0 1 1 2 0 1 2 2 2 2 0 2 0 0 2 0
1 0 1 2 2 1 2 0 2 0 2 1 0 1 2 0 0 0 0 2 2 0 0 2 2 2 1 1 2 0 1 2 1 0 1 1
2 2 0 0 1 1 1 1 0 0 0 0 0 2 0 2 0 1 0 1 0 0 0 0 0 2 0 2 1 1 0 1 0 0 1 2
2 0 0 2 0 1 2 2 2 0 2 2 1 1 0 0 0 0 0 0 0 0 1 0 1 0 2 0 1 0 1 1 2 1 0 0
2 1 2 0 2 0 0 0 0 0 0 1 2 0 1 0 1 0 2 0 1 0 0 0 1 1 0 1 0 0 1 2 0 2 0 2
2 0 0 2 1 0 2 2 0 1 0 1 2 1 2 0 1 0 2 0 2 1 0 2 2 0 2 0 2 0 2 0 0 0 0 0
1 0 1 1 0 1 0 1 2 0 0 2 2 0 0 1 0 0 0 2 2 0 2 1 2 0 1 2 2 0 1 1 0 2 2 0
0 0 1 2 0 1 1 0 0 1 1 0 0 1 0 2 0 2 1 0 0 2 2 0 2 2 0 0 0 0 2 1 0 1
1 0 0 0 0 0 2 0 1 1 2 0 1 0 0 2 1 0 0 1 1 2 1 2 2 0 0 0 0 0 1 0 0 1 0
0 0 2 0 1 1 0 0 0 1 2 1 2 0 2 1 0 0 0 0 2 0 2 0 1 2 2 0 1 0 1 2 0 2 0 1
1 0 0 0 2 2 0 0 2 2 0 1 2 2 0 0 0 0 1 2 1 0 2 0 2 0 0 0 1 2 0 0 0 0 0 1
0 2 0 2 0 1 0 2 0 0 0 2 0 0 0 1 0 2 0 0 1 2 1 0 0 0 1 2 0 0 0 2 0 2
0 0 0 1 1 2 1 2 2 0 2 0 2 0 2 0 1 2 1 1 0 0 1 2 1 1 0 1 2 1 1 0 2 0 0 1
0 1 2 0 1 0 0 0 1 2 0 0 2 2 1 2 1 0 0 0 0 1 2 0 0 0 0 0 1 1 0 0 0 0 1 0
1 2 2 1 2 2 1 2 1 0 1 0 0 1 0 0 2 1 0 0 2 2 0 2 0 2 1 2 1 1 1 0 2
2 2 1 0 0 2 2 2 1 2 1 0 0 0 2 1 0 2 0 2 1 0 0 2 1 0 1 1 1 0 1 2 2 1 0 0 0
0 2 0 0 2 0 0 1 2 1 2 1 0 2 2 1 1 0 1 1 0 2 2 1 0 2 1 0 0 0 0 0 1 1 0 2 0
0 1 2 0 2 0 1 1 0 0 0 2 0 1 2 2 1 2 0 0 1 2 0 1 1 0 2 1 0 2 0 1 0 0 0
0 1 0 1 1 0 0 0 0 2 0 1 1 2 0 0 2 0 0 0 1 2 1 1 0 2 2 0 1 0 0 2 2 1 0 0 1
2 0 0 0 0 0 0 2 2 0 1 1 0 0 0 0 1 2 1 2 2 2 2 1 1 2 1 2 0 0 0 0 2 1 0 0
2]
```

```
plt.scatter(X_pca[y_kmeans == 1,
0], X_pca[y_kmeans == 1, 1], s=100,
c='blue', label='Cluster 2')
plt.scatter(X_pca[y_kmeans == 2,
0], X_pca[y_kmeans == 2, 1], s=100,
c='green', label='Cluster 3')
plt.scatter(kmeans.cluster_centers_
[:,0], kmeans.cluster_centers_[:,
1], s=300, c='yellow',
label='Centroids (raw)')

plt.title('Clusters of Students
(PCA Projection)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.show()
```



Part 2 : Horizonital

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import scipy.cluster.hierarchy
as sch
from sklearn.cluster
import AgglomerativeClustering
from sklearn.decomposition
import PCA

dataset =
pd.read_csv('StudentsPerformance
.csv')
X = dataset.iloc[:, [5, 6,
7]].values # math score,
reading score, writing score
print(X)

dendrogram =
sch.dendrogram(sch.linkage(X,
method='ward'))
plt.title('Dendrogram')
plt.xlabel('Students')
```

```
[[72 72 74]
 [69 90 88]
 [90 95 93]
 ...
 [59 71 65]
 [68 78 77]
 [77 86 86]]
```

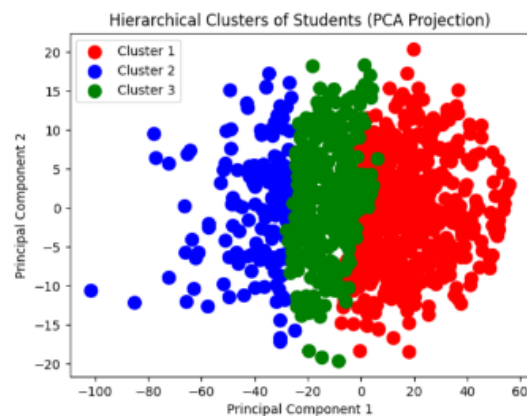
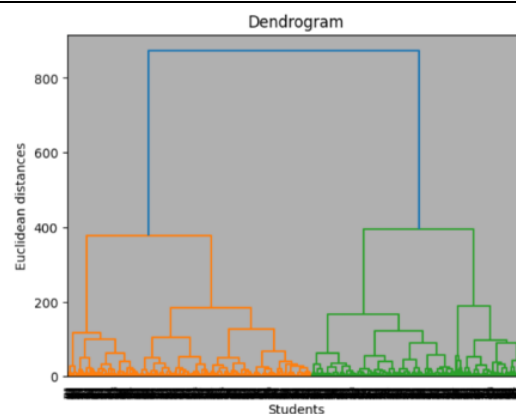
```
plt.ylabel('Euclidean
distances')
plt.grid(True)
plt.show()

hc =
AgglomerativeClustering(n_cluste
rs=3, metric='euclidean',
linkage='ward')
y_hc = hc.fit_predict(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

plt.scatter(X_pca[y_hc == 0, 0],
X_pca[y_hc == 0, 1], s=100,
c='red', label='Cluster 1')
plt.scatter(X_pca[y_hc == 1, 0],
X_pca[y_hc == 1, 1], s=100,
c='blue', label='Cluster 2')
plt.scatter(X_pca[y_hc == 2, 0],
X_pca[y_hc == 2, 1], s=100,
c='green', label='Cluster 3')

plt.title('Hierarchical Clusters
of Students (PCA Projection)')
plt.xlabel('Principal Component
1')
plt.ylabel('Principal Component
2')
plt.legend()
plt.show()
```



19- Explain the strategy for improving the system after viewing the cluster diagram.

Strategy for Improving the System After Viewing the Cluster Diagram

After analysing the cluster diagrams generated from the K-means and hierarchical clustering models, we can do the following solutions to improve the system and enhance students' performance:

CIS 2423 GroupProjectV1

1. Targeted Interventions Based on Clusters

- **Cluster 1 (Low Scores):**
 - **Issue:** Students in this cluster may struggle across all subjects (math, reading, writing).
 - **Solution:** Provide additional tutoring, personalized learning plans, or one-on-one teacher time.
- **Cluster 2 (Moderate Scores):**
 - **Issue:** Students perform moderately but may lack consistency in certain subjects.
 - **Solution:** Focus on strengthening weaker areas (e.g., math) through workshops or peer study groups.
- **Cluster 3 (High Scores):**
 - **Issue:** High performers may not be sufficiently challenged.
 - **Solution:** Offer advanced coursework or enrichment programs to keep them engaged.

2. Resource Allocation

- Allocate more resources (teachers, materials) to clusters with lower performance.
- Use clustering insights to design tailored teaching methods (e.g., visual aids for math, interactive reading sessions).

3. Early Warning System

- Implement predictive analytics to flag at-risk students early based on cluster trends.
- Monitor progress and adjust strategies dynamically.

4. Parental and Community Engagement

- Share cluster-based insights with parents to support collaboration in addressing student needs.
- Organize community workshops for skill-building (e.g., test preparation).

5. Curriculum Adjustments

- Modify teaching strategies based on cluster patterns (e.g., more practice tests for writing if Cluster 2 struggles).
- Introduce collaborative projects to bridge gaps between subjects.

6. Continuous Feedback

- Regularly update clustering models with new data to improve strategies.

Clustering helps teachers move from teaching everyone the same way to using data to support each student according to their need which would lead to more improvements.

CLO 4 - Version Control

1. Create a new repo for project in Git Hub

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner * Repository name *

 HesKal / Data-Analytics-Project1

✔ Data-Analytics-Project1 is available.

Great repository names are short and memorable. Need inspiration? How about [miniature-meme](#) ?

Description (optional)

ssion, classification, and clustering models built with Python and evaluated using machine learning techniques

- ☐  Public
Anyone on the internet can see this repository. You choose who can commit.
- ☒  Private
You choose who can see and commit to this repository.

Initialize this repository with:

- ☒ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs.](#)

2. Upload all the project files created for CLO1, CLO2 and CLO3 to the Git Hub repo

main	1 Branch	0 Tags	Go to file	Add file	Code
HesKal Upload project files including Word report and Colab notebook 6ab3820 · 19 minutes ago 2 Commits					
CIS 2423_Project_Report_202420.docx	Upload project files including Word report and Colab note...		19 minutes ago		
Project.ipynb	Upload project files including Word report and Colab note...		19 minutes ago		
README.md	Initial commit		34 minutes ago		

3. Configure Git with GitHub

```

receiving objects: 100% (7/7), 2.00 MiB | 12.75 MiB/s, done.
PS C:\Users\hkab5> git config --global user.email "H00535166@hct.ac.ae"
PS C:\Users\hkab5> git config --global user.name "HesKal"

```

4. Clone Git hub repo to Git

```

PS C:\Users\hkab5> git clone https://github.com/HesKal/Data-Analytics-Project.git
Cloning into 'Data-Analytics-Project'...
info: please complete authentication in your browser...
remote: Enumerating objects: 7, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (7/7), 2.00 MiB | 12.75 MiB/s, done.

```

5. Pull any file from Git Hub repo to Git

```

PS C:\Users\hkab5> cd Data-Analytics-Project
PS C:\Users\hkab5\Data-Analytics-Project> git pull origin main
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 1.89 KiB | 87.00 KiB/s, done.
From https://github.com/HesKal/Data-Analytics-Project
 * branch                main                -> FETCH_HEAD
    5deec9f..e05254c      main                -> origin/main
Updating 5deec9f..e05254c
Fast-forward
 README.md | 57 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++-----
 1 file changed, 55 insertions(+), 2 deletions(-)

```

6. Modify the pulled file and push the modified file to Git Hub

```
PS C:\Users\hkb5> cd Data-Analytics-Project
PS C:\Users\hkb5\Data-Analytics-Project> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   CIS 2423_Project_Report_202420.docx

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\hkb5\Data-Analytics-Project> git add "CIS 2423_Project_Report_202420.docx"
PS C:\Users\hkb5\Data-Analytics-Project> git commit -m "Updated CIS 2423 Project Report with latest changes"
[main d4ee7e1] Updated CIS 2423 Project Report with latest changes
1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\hkb5\Data-Analytics-Project> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 234.25 KiB | 3.60 MiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/HesKal/Data-Analytics-Project.git
6ab3820..d4ee7e1  main -> main
PS C:\Users\hkb5\Data-Analytics-Project>
```

Data-Analytics-Project
Private
Unwatch

main
1 Branch
0 Tags

Add file
Code

Commit	Author	Message	Time
d4ee7e1	HesKal	Updated CIS 2423 Project Report with latest changes	2 minutes ago
		CIS 2423_Project_Report_202420.docx	Updated CIS 2423 Project Report with latest changes
		Project.ipynb	Upload project files including Word report and Colab note...
		README.md	Initial commit

README

Summary And Conclusion

For this project, we looked at a dataset about student performance to figure out how things like background and outside factors affect how well students do in school. We did some basic data exploring (EDA), ran some tests, and even built a few machine learning models to try to predict student success.

The main idea was to find out if things like gender, race, parents' education, lunch programs, and test prep made a difference in students' math, reading, and writing scores. We wanted to see if we could discover patterns that might help teachers and schools support students better.

CIS 2423 GroupProjectV1

We used Python for the whole project. It's a programming language with tools like Pandas, NumPy, Matplotlib, Seaborn, and scikit-learn all of which made it easier to handle the data, make graphs, and build machine learning models.

We used a Logistic Regression model to predict whether students would pass or fail based on their scores. We also used linear regression to see if math and reading scores could predict writing scores.

We used both random and systematic sampling to pick students from the dataset. Then, we looked at basic statistics like the average scores, the most common scores, how spread out the scores were, and whether they leaned higher or lower.

- We used a chi-square test to check if gender and test prep were connected. It turned out they weren't really linked.
- We also did a one-sample t-test to see if the average writing score for girls was different from the overall average, but it wasn't.

We tested four models: Logistic Regression, KNN, Naïve Bayes, and Decision Tree. We used confusion matrices and checked things like accuracy, precision, recall, and F1-scores. Logistic Regression did the best overall.

We also tried grouping students into clusters using K-means and hierarchical clustering. That way, we could suggest different strategies for different types of students based on how they were doing.

We worked together as a team on this project. After all the exploring, testing, modeling, and clustering, we found some interesting insights about how outside factors can affect student success. Our prediction model worked well, and clustering gave us good ideas on how schools might better help different groups of students. Overall, it showed that using data can really help schools make smarter choices to support students.

Names	Tasks done
Maryam Ali	1- Define the purpose of data analysis for the chosen dataset 2- Identify and justify the type of programming used for data analysis 3- Identify the type and purpose of the machine learning algorithm to be implemented for the chosen dataset

	4- Identify and justify the independent and dependent variables for the chosen dataset. 5- Justify why you want to perform the descriptive analysis for the chosen dataset. 11- Perform the hypothesis test to find the correlation (Pearson and Spearman for numerical variable and chi-square test for categorical variable) between the independent variable and the dependent variable. 12- Assess the performance of the dependent variable to know whether the sample is representative of the normal population by a one-sample t-test. 15- Predict the value of the dependent variable from the different classifier such as Logistic Regression, KNN, Naïve-Bayes and Decision Tree. 19- Explain the strategy for improving the system after viewing the cluster diagram.
Nourah Abduallah	8- Create a script for systematic sampling by giving certain conditions and finding the desc stat for the dependent variable from the sample [Apply the descriptive function which you created]. 9- Create a detailed descriptive statistics report about the dependent variable of the chosen dataset. 13- Build, Train, Develop and Evaluate using Simple Regression for chosen dataset.

	17- Predict the dependent variable by using best-fit classifier.
Hessa khalfan	6. Create a script to develop a Python function for descriptive statistics. The input for the function should be the sample and the field to perform the descriptive statistics. 7. Create a program to random sampling of size 150 and find the descriptive statistics for the dependent variable from the sample [Apply the descriptive function which you created]. 16. Evaluate the performance of each model using confusion matrix and accuracy and identify the best fit classifier for the chosen dataset. 20. Create a new repo for project in Git Hub 21. Upload all the project files created for CLO1,CLO2 and CLO3 to the Git Hub repo 22. Configure Git with GitHub 23. Clone Git hub repo to Git 24. Pull any file from Git Hub repo to Git 25. Modify the pulled file and push the modified file to Git Hub
Meera Mohamed	• Data Cleaning 10- Visualize the dependent variable by the Graph/Chart of the following using Python Program 14- Develop a script to forecast the value of the dependent variable from all the relevant independent variables using Multiple Linear Regression



	18- Perform the cluster analysis such as K-means and Horizontal for any field from the chosen dataset. • Conclusion and Summary
--	--

Rubric

Please note that the Project rubric should reflect the project description and be CAP-compliant. Please feel free to customize the descriptors as per the project requirements and course level.

Group Project Rubric [Task-specific RUBRIC]

		Absent (F)	Insufficient (1-59%) (F)	Emerging (60-69%) (D/D+/C-)	Satisfactory (70-76%) (C/C+)	Competent (77-86%) (B-/B/B+)	Mastering (87-100%) (A-/A)
Group Grading- Critical Thinking: Analysis and Evaluation [45%]	Deliverable 1: understanding of data analytics [CLO1] (5%)		None of the following: - Purpose of data analysis not mentioned. - Selection of programming language is not referenced. - Type of machine learning algorithm to be analyzed is not defined - Variables for data analysis are not mentioned	Some but not all the following: - Purpose of data analysis mentioned but not clear. - Selection of programming language is referenced. - Required machine learning algorithm to be analyzed is ill-defined [Too many models mentioned] - Variables for data analysis are mentioned without justification	Most but not all the following: - Purpose of data analysis is clearly mentioned. - Selection of programming language is referenced. - Required machine learning algorithm to be analyzed is defined, but without justification. - Variables for data analysis are mentioned with justification but not clear	All of the following: - Purpose of data analysis is clearly mentioned. - Sound rationale for choosing the programming language - Required machine learning algorithm to be analyzed is well-defined with appropriate justification. - Variables for data analysis are mentioned with clear justification	All of the following: - Purpose of data analysis is clearly mentioned with appropriate explanation - Thoroughness of programming language chosen is evident. - Student synthesizes information about machine learning algorithm from multiple disciplines and sources and presents them with clarity. - Variables for data analysis are precisely defined with appropriate justification

							• Employ outstanding knowledge about the data analysis from different sources
	Deliverable 2: Apply data analysis and visualization techniques [CLO2] (25%)		• No evidence of justification to perform descriptive statistics • Function is not created for descriptive statistics • Sampling techniques are not designed • Exploratory analysis are not performed • Hypothesis tastings are not achieved • Data preprocessing is not devised • Regression models are not functioned • Optimization for regression model is not operated • Best fit model is not enacted	Some but not all the following: • Evidence of justification to perform descriptive statistics exist but not clear • Function is created for descriptive statistics having major syntactical errors. • Sampling techniques are designed but not appropriate • Exploratory analysis are performed but ill-designed • Hypothesis tastings are achieved with flaw • Evidence for data preprocessing exist but with inappropriate techniques	Most but not all the following: • Evidence of justification to perform descriptive statistics exist with clear elucidation • Function is created for descriptive statistics without flaws. • Sampling techniques are designed with appropriate rationalization • Exploratory analysis are performed with appropriate legends • Hypothesis tastings are achieved appropriate assumption • Evidence for data preprocessing exist but with applicable techniques	All of the following: • Appropriate description and exploratory analysis provided. • Sound rationale for chosen hypothesis testing • High accuracy obtained for regression model • Rigor is evident.	All of the following: • Appropriate description and exploratory analysis provided along with exemplary analysis • Sound rationale for chosen hypothesis testing and infer statically in commendable way • Other regression model are handled obtained from different sources • Rigor is evident.

				- Regression models are functioned with huge faults - Optimization for regression model is operated with inaccuracies Best fit model is enacted without justification.	- Regression models are functioned without mistake - Optimization for regression model is operated with appropriate metrics - Best fit model is enacted appropriate interpretation		
	Deliverable 3: The data model created using classification and clustering algorithm of machine learning [CLO2, CLO3] (15%)		- Incomplete classification models are presented. - No optimization and recommendation are performed for classification model. - No evidence of any cluster analysis - No strategies are produced from cluster model.	Some but not all the following: - Comprehensive classification models are presented but with significant errors - Thorough optimization and recommendation are performed for classification model but with poor validation - Evidence of cluster analysis exists with poor performance - Strategies are derived from cluster model which are insignificant	Most but not all the following: - Comprehensive flawless classification models are presented - Thorough optimization and recommendation are performed with appropriate validation for classification model - Evidence of cluster analysis exists with better performance - Strategies are derived from cluster model which are significant but not consistent	All of the following: - Implemented all classification models - Major validation works demonstrated on optimization and recommendation of classification model - Appropriate cluster analysis was performed and good strategies are postulated and well-documented.	All of the following: - All classification models are accurate - Perform more classification models from different sources - All cluster models are accurate - Perform more cluster models from different sources - Appropriate interpretation and recommendations are provided

	Deliverable 4: Data Versioning [CLO 4] (5%)		• No evidence of creating a new repository and uploading project files to Git Hub • Git is not configured with Git Hub • No evidence for cloning Git hub repo to Git • Push and Pull are not performed	Some but not all the following: • Repository model is created but files are not uploaded to repo • Git is not configured with Git Hub • Evidence of cloning exists with appropriate options • Push and Pull are performed which are insignificant	Most but not all the following: • Repo are created and project files are uploaded to repo • Git configuration are performed with accordance of Git Hub • Evidence of cloning exists with proper options • Push and Pull are significant but not consistent	All of the following: • Repo are created and project files are uploaded • Advanced configuration is performed on Git • Many functions related to cloning is performed apart from basic operations • Push and pull with different scenario are mentioned	All of the following: • Scenario is created and explained about significance of repo creation • Advanced configuration is performed on Git • Different cloning techniques should be elucidated. • Push and pull with different scenario are mentioned • Branching should be exhibited
Group Grading: Report Quality [5%]			• Incomplete report with missing most of the deliverable components. • Too many typographical errors.	Some but not all of the following: • Complete report with required deliverables. • Clear table of contents showing all required sections. • Free from formatting and typographical errors.	Most but not all of the following: • Complete report with all required format and deliverables. • Some minor formatting and/or typographical errors. • Table of contents presented with some missing information.	All of the following: • Complete report with required format and deliverables. • Clear table of contents showing all required sections. • Some minor formatting and/or typographical errors.	All of the following: • Complete report with required format and deliverables. • Clear table of contents showing all required sections. • Free from formatting and typographical errors.

Individual Oral Presentation

Student (1) Name: _____

Student HCT ID: _____

	Absent	1-59.49%	59.5% - 69.49%	69.5% - 76.49%	76.5% à86.49%	86.5% à100%
Collaboration (10%)		- Does not partake in the task or is so frequently distracted that he/she produces. - Having made no contribution to the group's objective.	- Participates in the task but does not collaborate with others or contribute to the group process. - Participates in discussions. - Expresses own opinion and viewpoints. - Remains dedicated to the subject. - Performs independently specific tasks	- Cooperates with the group process, but does not coordinate contributions with those of others. - Does not interrupt while listening. - Actively seeks the input of others. - Accepts duties that have been assigned. - Complies with the group consensus. - Utilizes the ideas of others	- Coordinates processes and products with those of teammates, but does not resolve significant conflicts. - Actively pays attention to what is being said. - Offers and accepts constructive feedback. - Adapts ideas and/or processes to the needs of colleagues. - Seeks consensus. - Effectively resolves trivial disputes	- Student coordinates collaborative processes and outputs. - Effectively resolves both significant and minor conflicts. - Expresses disagreements in an open and diplomatic manner. - Supports group decisions even if they are not unanimously supported. - Compromised and negotiated to attain an agreement

Oral Communication (Presentation) (12 Marks) [All CLOs]		- Communicates with a limited sense of audience and purpose - No eye contact, no body language, and no decorum - Communicates with limited clarity - Uses language with limited accuracy and effectiveness. - Tension and anxiety are palpable; - Has difficulty recovering from errors.	Not all of the foregoing are true: - Communicates with a clear sense of audience and purpose - Eye contact, body language, and decorum) - Communicates information and ideas with considerable clarity - Uses language with considerable accuracy and efficacy - Makes insignificant errors, but rapidly recovers - Displays minimal or no tension.	Most of the following, but not all: - Communicates with a clear sense of audience and purpose - Eye contact, body language, and decorum) - Communicates information and ideas with substantial clarity - Uses language with substantial accuracy and effectiveness. - Makes simple mistakes but recovers swiftly; - Displays minimal or no tension.	Each of the subsequent: - Communicates with a clear sense of audience and purpose - Eye contact, body language, and decorum - Communicates information and ideas with considerable clarity - Uses language with considerable accuracy and efficacy. - Makes insignificant errors, but recovers rapidly - Demonstrates minimal or no tension.	Each of the subsequent: - Communicates with a strong awareness of audience and purpose - Maintains audience's attention by using direct eye contact and rarely glancing at notes. - Communicates information and ideas with a high degree of precision and precision - Student appears at ease and confident, with no errors.
--	--	---	---	---	--	---

Follow-up questions and discussion (28 Marks) [All CLOs]	Unable to answer questions from the examining board	Able to answer some but not all questions from the examining board	Able to answer most but not all questions from the examining board	Capable of answering all of the examining board's queries.	Capable of responding to all inquiries and demonstrating a thorough understanding of the material.
---	---	--	--	--	--